By supporting rich entities and strong typing, Cedar promotes structure rather than ad hoc logic, making it easier to write policies that are both correct and understandable. It also provides a foundation for advanced analysis tools to identify coverage gaps, policy conflicts, or unexpected grants.

7.4.3 Clear namespace and type safety

Cedar organizes all principals, resources, and actions using namespaces, which appear before the `::` delimiter. For example:

```
Finance::Document
CustomerCollab::Employee
```

Namespaces have several benefits:

- Disambiguate entities and actions—Imagine ACME manages both a furniture inventory service and a general-purpose database system, each having a `Table` entity type. Namespaced references like `Furniture::Table` and `Database::Table` prevent conflicts when policies are grouped or merged.
- Organize policy domains clearly—By grouping related entities and actions under distinct namespaces, Cedar helps teams maintain clear boundaries between application areas more easily.
- Control visibility and access — Namespaces naturally set boundaries: Types are unqualified within their namespace but need to be fully qualified when referenced across different namespaces. This enhances clarity and enforces the desired access control scopes.

Namespaces also prevent incorrect attribute expressions, such as `resource.owner`. Since each entity type is defined within a namespace, Cedar ensures that `owner` exists on that specific type. Using `resource.owner` on an entity without that attribute results in a type error before deployment.

Likewise, namespaces ensure type-safe policy logic: You can’t mistakenly compare an Employee to a Document or perform an IP check on a timestamp. Namespaces combined with attribute definitions strengthen the